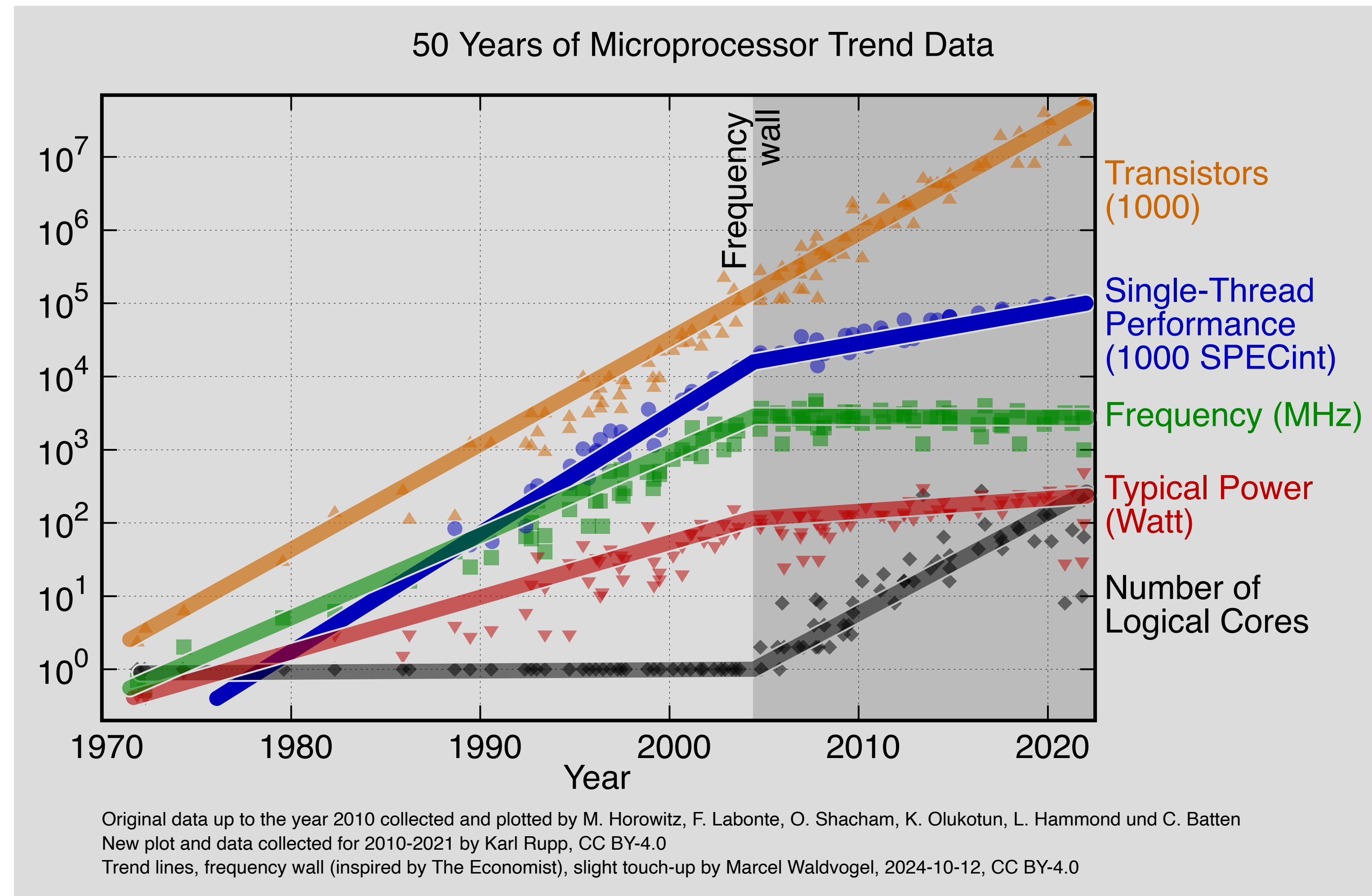


Processor Architecture of Modern Systems

What do we do with the transistors?

Where are we now?

- Transistor count still growing by Moore's law.
- Clock speed flat for 20 years.
- Single-thread SPECint performance slowing increasing, but not remotely close to transistor count.



What are the blockers?

RAM Latency

- DDR5-8800 CL36 RAM is about the best you can get in 2025, and it has a first-word latency of about **8ns**.
- Contrast PC100 memory from the early 1990s, which would have a latency of the order of **10ns**.
- Contrast async RAM of the 1980s, about **35ns**.
 - **Essentially, no change in 30+ years**, as clock speeds on processors have gone from 15MHz to 3GHz, 200x.
 - 15MHz = ~65ns per clock cycle, RAM is “fast enough”
 - 3GHZ = ~0.3ns per clock cycle, RAM is 24 times slower than the processor clock.
- <https://ramlatencycalculator.xyz/>

What are the blockers?

Frequency Wall

- It's hard to build microprocessors with high(er) clock speeds, because of power consumption, speed of light / propagation issues, capacitance, a variety of issues. 20 years of work haven't fixed this.
 - With a 3GHz clock, electricity travels less than 9cm in one clock cycle
- Higher clock speeds make design of the overall package harder, as well as the design of the die.
- Power consumption scales badly with frequency, which in turn brings cooling problems.

What can we do with the transistors?

Option 1: more cache

- We can add more cache to the processor, to decouple from the RAM performance.
- But as we saw last week, cache hit rate is asymptotic, so past a certain point it doesn't buy us much.
- SRAM is quite power-hungry, and we have to locate it close in to the CPU core in order to get the full performance.
- Cache hit rates are probably, realistically, as good as they are going to get.

What can we do with the transistors?

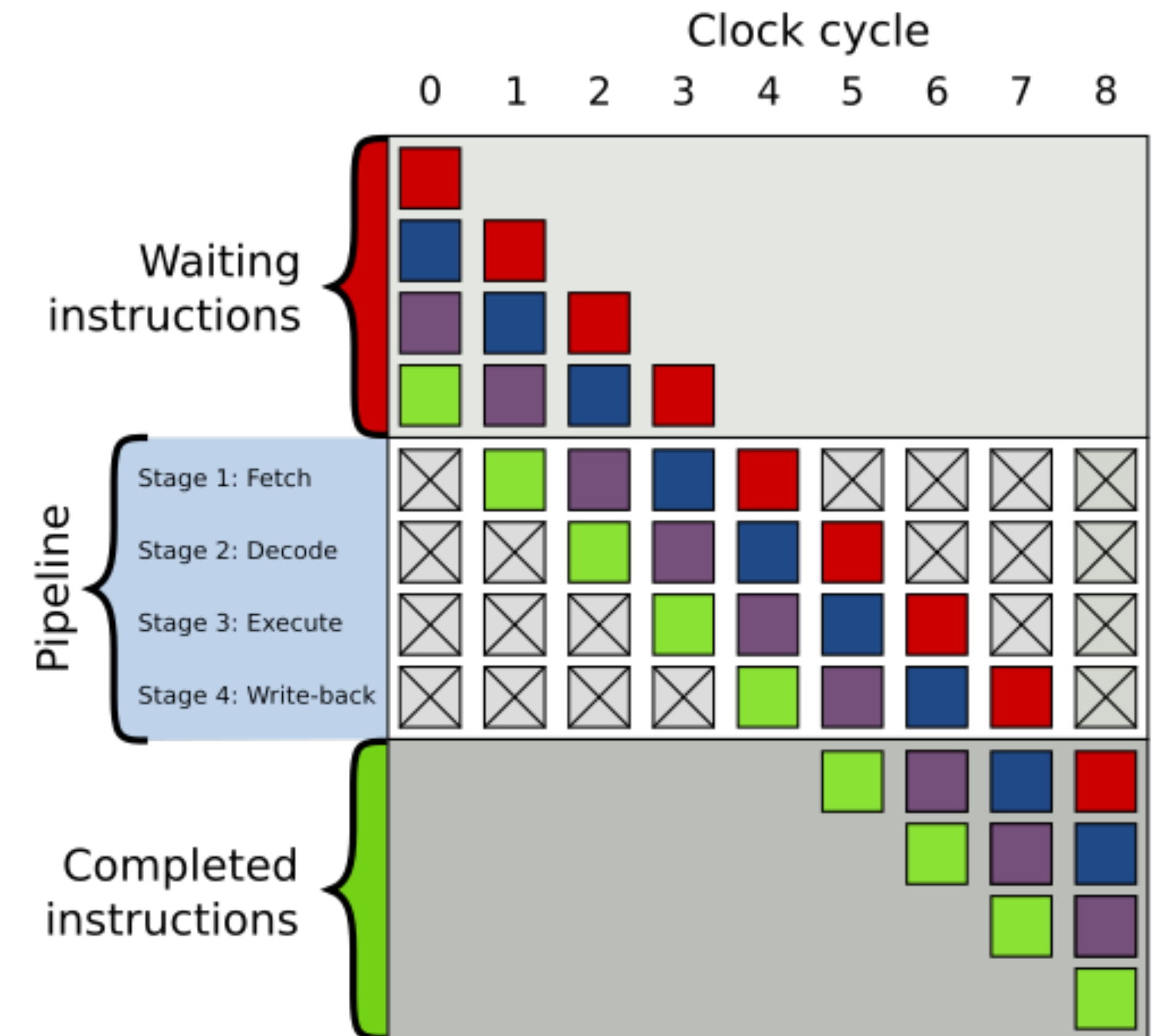
Option 2: more complicated execution

- We can make a processor more complicated, in some vague sense, so that it does more on each clock cycle or more with each instruction. More complicated addressing modes, more complex instructions, more registers, more instructions that process lots of data at a time.
- Often workload-specific (think GPUs)
- Track record is not good
- Also increases chances of flaws in processors

What can we do with the transistors?

Option 3: More instructions in flight (“pipeline”)

- An individual processor has multiple execution units
- Perhaps we can decode the next instruction while we are processing the current one?
- Perhaps we can fetch all the operands for the next instruction while we are processing the current one? And maybe decode the next instruction after that?



What can we do with the transistors?

Option 4: more threads

- Rather than pipeline instructions on one thread, we can use the execution units to run multiple threads, which map onto multiple threads in programs.
 - pipelining: run a single thread faster, by overlapping execution on available units.
 - simultaneous multithreading: run n threads on $<n$ cores, using the elements that are available in a more flexible way.
 - SMT requires you actually have a threaded workload, which may not be the case.

What can we do with the transistors?

Option 5: more processors

- We can have multiple processors, perhaps sharing cache, perhaps not, perhaps sharing RAM, perhaps not, perhaps on the same package, perhaps not.
- Most operating systems were modified to run on multiple processors (“symmetric multi-processing”) in the 1990s and will manage to run separate programs on separate processors reasonably efficiently.

Today's approach

All of the above

- Each processor core will be deeply pipelined, so that fetches of RAM can happen in advance
- Each processor core may run more than one thread
- There will be more than one core in each package
- There may be more than one package
- The operating system will very probably see multiple processors equal to threads time cores times packages.
- All processors beyond the simplest are pipelined (“super-scalar”, although they are not strictly synonyms)

So what are the issues?

- There are two aspects to a processor architecture, which we might call the “macro” and the “micro” architecture. The “macro” architecture is the high-level structure we saw in the previous lectures. The “micro” architecture is the messy reality, with instructions being executed out of order, in pipelines, speculatively, etc.
- The desired goal is making the micro architecture consistent with the macro architecture, so that programmers can essentially treat the macro architecture as all there is.

Let's look first at pipelining

- We see pipelining in many contexts.
- When we are sending network traffic, we cannot stop and wait after every packet, we need to send enough to need to network busy.
- When we are manufacturing things, we do not wait for things to reach the end of the production line before we start the next one.
- Just because a 4-engined bomber left the production line at Willow Run every 62 minutes does not mean it only took 62 minutes to build a four-engined bomber.



Pipelining sounds easy

- While we are writing the results of one instruction back to memory...
- We are executing the next instruction
- And decoding the instruction after that
- And fetching the instruction after that
- If we have RAM which is ~25 clock cycles of latency, ideally the pipeline would be at least fifty stages long. Then we could complete an instruction on every clock cycle (assuming a fetch, and an instruction format involving only one trip to main memory).

So what's the problem?

- If we had hundreds of instructions in a “straight line”, so that from the perspective of one instruction we could reliably know the instruction we will execute fifty or more instructions in the future, it would all be good.
- How realistic is this?
- Think of a typical program: it contains loops, conditionals, function calls that may or may not happen, etc.
- And there is a lot of “housekeeping” in a real computer system: page faults, interrupts, etc.

In passing, loop unrolling

- If you have loops which have constant bounds
 - `for (i = 0; i<100; i++) { a[i]=i; }`
- You can unroll them
 - `a[0] = 0; a[1] = 1; a[2] = 2; a[3] = 3...`
- The trade-offs aren't simple, but compilers will sometimes do this.
- In any event, many/most loops have more complex conditions on them.

Conditional Branches kill this

- Whenever we branch in a way which depends on prior execution or events, we impact our ability to predict the next and subsequent instructions.
- What can we do to mitigate this?

A simple solution: deferred branch

- 1990s saw a rise in “RISC” processors: Reduced Instruction Set Computers. Less charitably, “Relegate Interesting Stuff to the Compiler”
- Examples: IBM 801, MIPS, SPARC, Alpha, Motorola 88k, ARM, AMD 29000, Power, i860, i960...every manufacturer in every niche.
- Characterised by fixed length instructions, simpler addressing modes, lots of registers, and optimisation for pipelining.
- Deferred Branch was an early attempt

Deferred Branch

- If we branch conditionally, we will probably need to pause fractionally because until the condition is executed, we don't know what to fetch next.
- Deferred branch makes a branch look like this:
 - Test whether two registers are equal by subtracting them
 - Branch on Zero
 - Always do this next instruction

Deferred Branch

- Sometimes this makes sense, and the compiler can do something sensible:
- `for (i = 0; i < 100; i++) { array[i] = i; }`
 - Load 0 into A
 - loop: Store A to array+A
 - Copy A to B
 - Subtract 100 from B
 - Branch to “loop” if negative
 - add 1 to A // do this while we are re-fetching loop

Deferred Branch

- Throws a lot of load onto the compiler (gcc, the dominant compiler of the period, for many years always just put a NOP into the delay slot)
- Although the contrived example above looks somewhat convincing, simply adding one extra instruction still leaves any pipeline of more than one slot nearly empty (“stalled”) when the branch happens.
- We need a more general solution

Branch Prediction

- A lot of the time, we can predict which way a branch will go. Not perfectly, but “most of the time”.
- For example, in a for/while/until loop, it’s reasonable to assume that you jump back to the start of the loop ($n-1$ times) more often than you jump out of the loop (1 time).
- If you have a system call followed by an if statement covering the error case, it’s a reasonable expectation that the error case is unusual.
- And of course, you can always instrument the code and find out experimentally.

Branch Prediction

- We can make this the programmers' problem, and introduce constructs like:
 - `if (likely(condition)) { ... something that normally happens ... }`
 - `if (unlikely(condition)) { .. something that rarely happens ... }`
- This allows the compiler to start fetching and processing instructions along the likely route rather than the default route
- In practice, this prediction is done both by the compiler and by the processor itself, so having the programmer do it (often badly) is not often recommended.
- Much better branch prediction is amongst the many reasons llvm has overtaken gcc as the preferred (one might joke “go-to”) compiler suite.

Speculative Execution

- Obviously, fetching instructions is a neutral act: either the instruction is then executed (good) or it's not (execution is no worse off).
- Similarly decoding instructions: either it gets executed, in which case it's faster, or it doesn't, which does us no harm.
- What about fetching data?
- What about performing calculations?
- In the limit, you can essentially execute down both paths of a branch, so long as you do not store data upon which other calculations depend.

Speculative Execution

- There are constraints
 - You need to make sure that executing instructions speculatively which it later turns out shouldn't have been executed (for example, the else side of an if statement where the condition was true) doesn't affect later execution.
 - This all assumes that the speculative execution does not consume resources that might otherwise be used more productively.

Types of Speculative Execution

- Eager execution: always execute both sides of a branch (and perhaps even all four sides of two successive branches)
- Branch Prediction: use some metric to predict which way the branch will go, but execute both sides when you're not sure.
- Many techniques: machine learning, feeding back past runs, caching branches (it's not unreasonable to assume that if a branch went one way last time, it'll go the same way this time, cf. loops)

Direct Cache

- Lurking in the background, we have caches.
- A cache is a copy of memory, but much faster.
 - At its simplest, take the address modulo the size of the cache, and store the memory contents and the rest of the address (tag)
 - When accessing memory, take the address modulo the size of the cache, and check whether the rest of the address matches.

10	14	34	19
----	----	----	----

- Cache: 256 entries
- Address: 0x10143419
- $0x10143419 \% 256$ (or $0x10143419 \& 0xff$) $\Rightarrow 19$
- Put it in slot 19 in the cache, with tag 0x101434

Set-Associative Cache

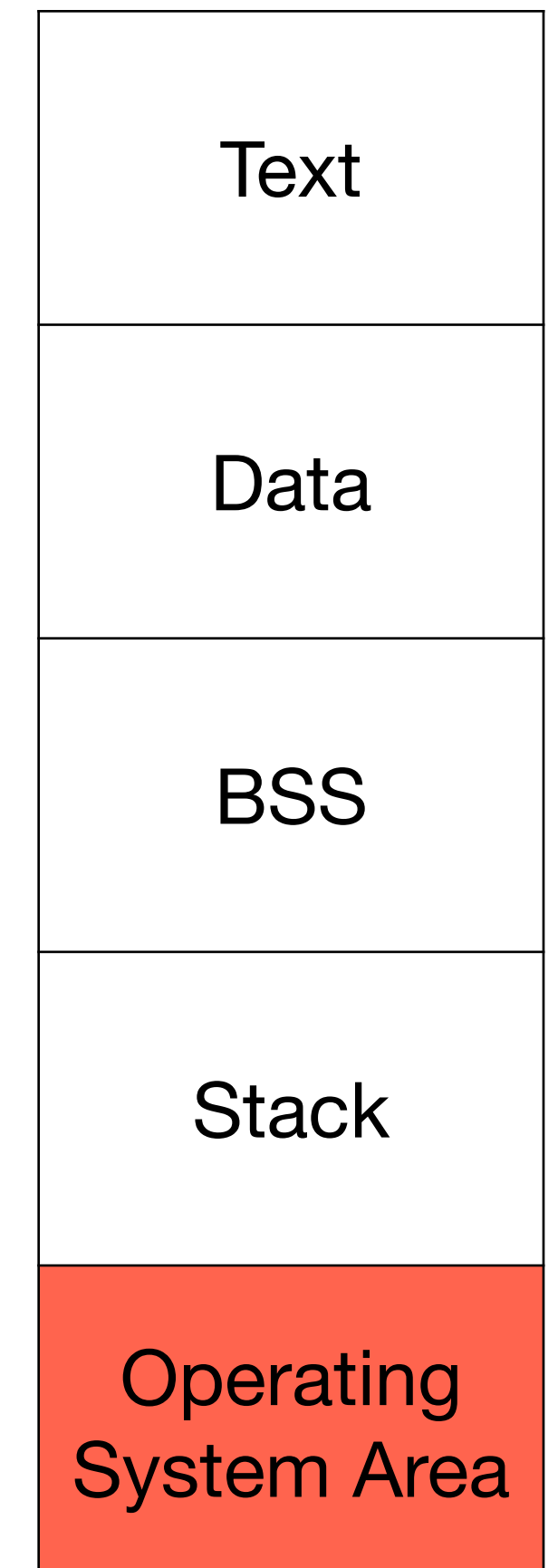
- Instead of the unit of the cache only storing one row at a time, store 2 (4, 8, 16) rows instead.
- Take the address modulo the number of units, and store the tag and the value in any available row
- Look up by taking the address modulo whatever, and searching for any matching tag
- Improves cache hit rate (marginally) at the expense of a more complex lookup.

How can we tell if our data was cached?

- Timing
- If the data was already in the cache, a fetch will be very quick.
- If it has to come from main memory, it will take longer.
- Basic intuition: read the system clock (probably around 1ns resolution). Fetch the contents of a memory location. Read the clock again. ~ 1 ? It came from cache. >1 ? It had to come from RAM
- Hold this thought.

Macro meets micro

- Some operating systems map operating system data into the address space of processes, but make it only available when in supervisor mode.
- It gives a useful place to put per-process information
- An example might be encryption keys: the operating system is doing encryption stuff for you, and manages the keys.
- [[This is not an OS class, and “how Docker and Virtualisation works” is a complex question, but “stuff the OS does on your behalf” covers a lot of ground.]]



Using Cache to Fetch Forbidden Data

- There is usually an instruction of the form “add this register to the contents of this slot in main memory, so I fetch the result of that calculation”
- Load Effective Address
- Intuition: `array[index]`
 - Load the address of array
 - Add index
 - Now fetch the result

LEA

LOAD

How to abuse it

	Macro	Micro
Allocate an array of 65536 elements. Call the address of element zero “base”.		
Identify 16 bits I want to read but am forbidden from accessing. Call the address of those bits “target”.		
LEA A0, base+(target) <i>Load into A0 base + contents of target,</i>	Fails: cannot read target	Runs, pending access check
LOAD (A0) <i>Load what A0 points to, MS[A0]</i>	Never Executed	Results discarded when previous fails

The Macro View

- An attempt to calculate an address using the contents of forbidden memory fails, there and then. The end.
- Everything that follows from that also fails, as no address was calculated.

The Micro View

- In a deep instruction pipeline, load instructions are being speculatively executed, and in parallel with that the checks are being made on whether they are permitted.
- It's OK to calculate $\text{base} + (\text{target})$, so long as we never show the result to anyone.
- It's OK to to fetch a value whose address has been calculated without proper checks, so long as we throw the results away if the previous instruction is found to have been illegal (everything “after” an illegal instruction should never have run).

How do we exploit this?

- Fetch every element from array, each access wrapped in a timer. Slow ones: that's not the value of target. The fast one? That's the value of target.
- Performance will depend on platform, but on average this will take under a millisecond, potentially **well** under a millisecond.
- What's going wrong?

What's going wrong?

- The problem is that the cache contents are being updated by speculative memory access, even though the access is never shown to the macro architecture.
- The fixes are major performance invasions: every time an instruction is speculatively executed but ultimately not “retired” (the results actually used), you need to flush some/most/all of the cache.
- But 99% of the time, you are fine with the cache, and cache flushes are not fast in and of themselves.

Specifics v Principles

- This is the basis for 2017's "Spectre" attacks.
- As well as the specific, it highlighted the general problem of "clever" optimisations which have unintended side-effects.
- In passing, there's a history back to 2000-ish of attacks on crypto based on timing, and "constant time" implementations (which run in the same time, irrespective of key and data) are a hot topic.



Is there more to come?

- Well, there's your PhD
- The disconnect between the idealised view of a processor, what you can see via the published interfaces of the actual processor, and what you can get by misusing the interfaces.
- Processors have become savagely complex, and the whole world of pipelining causes both performance peculiarities, and potentially security problems.

Multicore Processors

- We have had multiprocessor machines for a very long time: plenty of 1960s mainframes and 1970s minicomputers had multiple processors. Multiprocessor machines based around microprocessors naturally followed.
- They range from “clusters” (where you have multiple processors running multiple operating systems, but sharing storage) through various systems where one processor is in some sense controlling the others (one processor for O/S, others for applications) through to symmetric multiprocessors (all the same processor, one operating system image).
- Most systems now are SMP (symmetric multiprocessing).
- Few processors now are single processors: even a Pi has four processors on the die.

Ivan Sutherland's Cycle of Reincarnation

- A processor is doing a everything, with an operating system. “Uniprocessing”.
- We add another processor, to do something else, with a more specific software stack: network acceleration, graphics acceleration, vector processing, database processing... “Asymmetric Multiprocessing”.
- Over time, that processor becomes more powerful, and the operating system closer to the primary operating system. We end up with a system containing multiple instances of the same processor. “Symmetric Multiprocessing”.
- Processors get faster, and it's easier to have one fast processor than two slower ones. Now we're back to “Uniprocessing”, ready to start the cycle again.

Multiprocessing Efficiency

- If we have more than one processor, how much more does it get us?
 - We can probably schedule processes onto the processors, so if we have enough application work we can keep them busy.
 - Cache synchronisation between processors can be very difficult
 - Splitting I/O and other system activity between processors is difficult (Solaris got this right, after about ten years, Linux is getting there)
 - We also always prefer 2x in one processor than 2 processors of power x.

